МИНОБРНАУКИ РОССИИ

федеральное государственное автономное образовательное учреждение
высшего образования

**«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)»**

Кафедра «Информационных технологий и вычислительных систем»

СЕМИНАРЫ ПО ДИСЦИПЛИНЕ «БАЗЫ ДАННЫХ»

Семинары 3–6

Дано, теория, задачи, иллюстрации

Автор:
Семенов А. С.

Москва, 2026 год

Содержание

1 Семинар 3. Аренда жилья (варианты 1, 2) 2
1.1 Теоретическая справка . . . 2
1.2 Дано (стартовый набор данных) . . . 2
1.3 Задачи . . . 4
1.4 Решения . . . 4

2 Семинар 4. Подзапросы, EXISTS, оконные функции, CTE 6
2.1 Теоретическая справка . . . 6
2.2 Дано (стартовый набор данных) . . . 6
2.3 Задачи . . . 9

3 Семинар 5. Транзакции, индексирование, статистика, план выполнения 10
3.1 Теоретическая справка . . . 10
3.2 Дано (стартовый набор данных) . . . 10
3.3 Задачи . . . 12

4 Семинар 6. Проектирование БД, нормализация, DDL, ограничения, функции и триггеры 13
4.1 Теоретическая справка . . . 13
4.2 Дано (стартовый набор данных) . . . 14
4.3 Задачи . . . 16

1 Семинар 3. Аренда жилья (варианты 1, 2)

1.1 Теоретическая справка

Теория.

Реляционная модель данных: данные представлены в виде взаимосвязанных отношений (таблиц).

- **Отношение (таблица)** — множество кортежей одной структуры.
- **Кортеж (строка)** — одна запись.
- **Атрибут (столбец)** — поименованная характеристика.
- **Домен** — множество допустимых значений атрибута.
- **Первичный ключ (PK)** — однозначно идентифицирует кортеж.
- **Внешний ключ (FK)** — ссылается на PK другого отношения.

Типы соединений.

Тип JOIN	Что возвращает	Когда использовать
`INNER JOIN`	Только совпадающие строки	Только связанные записи
`LEFT JOIN`	Все строки левой + совпадения	Все записи левой таблицы
`RIGHT JOIN`	Все строки правой + совпадения	Симметрично LEFT
`FULL JOIN`	Все строки обеих	Все записи с двух сторон
`CROSS JOIN`	Декартово произведение	Генерация комбинаций

Агрегатные функции.

Функция	Назначение	Пример
`COUNT(*)`	Число строк	`COUNT(*)`
`COUNT(col)`	Число не-NULL	`COUNT(Phone)`
`SUM(col)`	Сумма	`SUM(MonthlyCost)`
`AVG(col)`	Среднее	`AVG(Area)`
`MIN(col)`	Минимум	`MIN(RentDate)`
`MAX(col)`	Максимум	`MAX(MonthlyCost)`

1.2 Дано (стартовый набор данных)

Дано (стартовый набор данных).

Создайте таблицы и заполните их данными. Имена таблиц и полей — латинские; русские названия и перевод значений приведены ниже.

```sql
-- Domain: "Rent housing"
CREATE TABLE Families (
    FamilyID   INTEGER PRIMARY KEY,
    FamilyName TEXT,
    HeadID     INTEGER,
    HeadName   TEXT
);

CREATE TABLE Realty (
    RealtyID INTEGER PRIMARY KEY,
    Address  TEXT,
    Area     INTEGER
);

CREATE TABLE Rent (
    RentID      INTEGER PRIMARY KEY,
    RentDate    DATE,
    FamilyID    INTEGER REFERENCES Families(FamilyID),
    RealtyID    INTEGER REFERENCES Realty(RealtyID),
    MonthlyCost INTEGER,
    Duration    TEXT
);
```

```sql
INSERT INTO Families VALUES
(201,'Ivanovy',301,'Ivan Ivanov'),
(202,'Petrovy',302,'Anna Petrova'),
(203,'Smirnovy',303,'Olga Smirnova'),
(204,'Kuznetsovy',304,'Sergey Kuznetsov');

INSERT INTO Realty VALUES
(1,'ul. Lenina, 10',70),
(2,'ul. Gagarina, 5',100),
(3,'ul. Molodezhnaya, 7',60),
(4,'ul. Pushkina, 3',85);

INSERT INTO Rent VALUES
(101,'2024-08-01',201,1,50000,'12 months'),
(102,'2024-08-02',202,2,80000,'6 months'),
(103,'2024-09-01',203,3,45000,'24 months'),
(104,'2024-09-15',204,4,70000,'12 months');
```

Перевод значений: `Ivanovy` — Ивановы, `Petrovy` — Петровы, `Smirnovy` — Смирновы, `Kuznetsovy` — Кузнецовы; `ul. Lenina, 10` — ул. Ленина, 10; `ul. Gagarina, 5` — ул. Гагарина, 5; `ul. Molodezhnaya, 7` — ул. Молодежная, 7; `ul. Pushkina, 3` — ул. Пушкина, 3.

Таблица `Families`**:**

FamilyID (PK)	FamilyName	HeadID	HeadName
201	Ивановы	301	Иван Иванов
202	Петровы	302	Анна Петрова
203	Смирновы	303	Ольга Смирнова
204	Кузнецовы	304	Сергей Кузнецов

Таблица `Realty`**:**

RealtyID (PK)	Address	Area
1	ул. Ленина, 10	70
2	ул. Гагарина, 5	100
3	ул. Молодежная, 7	60
4	ул. Пушкина, 3	85

Таблица `Rent`**:**

RentID (PK)	RentDate	FamilyID (FK)	RealtyID (FK)	MonthlyCost	Duration
101	2024-08-01	201	1	50000	12 months
102	2024-08-02	202	2	80000	6 months
103	2024-09-01	203	3	45000	24 months
104	2024-09-15	204	4	70000	12 months

1.3 Задачи

1. **1.1.** Список всех семей.
2. **1.2.** Список всех объектов недвижимости.
3. **2.1.** Общая сумма аренды за август 2024.
4. **2.2.** Стоимость аренды с датой 02.08.2024.
5. **3.1.** Фамилии семей и адреса их квартир.
6. **3.2.** Средняя площадь квартир по семьям.
7. **4.1.** Адрес квартиры с самой дорогой арендой.
8. **4.2.** Общая стоимость аренды за август 2024.
9. **5.1.** Семьи, арендующие квартиры площадью > 80 кв.м.
10. **5.2.** Дата первой аренды каждой семьи.

1.4 Решения

```sql
SELECT * FROM Families;                    -- 1.1

SELECT * FROM Realty;                      -- 1.2

SELECT SUM(MonthlyCost) AS TotalCost       -- 2.1 / 4.2
FROM Rent
WHERE RentDate BETWEEN '2024-08-01' AND '2024-08-31';

SELECT MonthlyCost FROM Rent               -- 2.2
WHERE RentDate = '2024-08-02';

SELECT F.FamilyName, R.Address             -- 3.1
```

```
FROM Rent T
JOIN Families F ON T.FamilyID = F.FamilyID
JOIN Realty   R ON T.RealtyID = R.RealtyID;

SELECT F.FamilyName, AVG(R.Area) AS AvgArea -- 3.2
FROM Rent T
JOIN Families F ON T.FamilyID = F.FamilyID
JOIN Realty   R ON T.RealtyID = R.RealtyID
GROUP BY F.FamilyName;

SELECT R.Address                            -- 4.1
FROM Realty R
JOIN Rent T ON R.RealtyID = T.RealtyID
WHERE T.MonthlyCost = (SELECT MAX(MonthlyCost) FROM Rent);

SELECT F.FamilyName                         -- 5.1
FROM Rent T
JOIN Families F ON T.FamilyID = F.FamilyID
JOIN Realty   R ON T.RealtyID = R.RealtyID
WHERE R.Area > 80;

SELECT F.FamilyName, MIN(T.RentDate)        -- 5.2
       AS FirstRentDate
FROM Rent T
JOIN Families F ON T.FamilyID = F.FamilyID
GROUP BY F.FamilyName;
```

Ожидаемый результат:

2.1 / 4.2: 130000. 2.2: 80000. 3.2: 70, 100, 60, 85. 4.1: ул. Гагарина, 5. 5.1: Петровы, Кузнецовы. 5.2: 2024-08-01, 2024-08-02, 2024-09-01, 2024-09-15.

2 Семинар 4. Подзапросы, EXISTS, оконные функции, CTE

2.1 Теоретическая справка

Теория.

Подзапрос — запрос внутри другого запроса: скалярный, табличный (`IN (SELECT ...)`), коррелированный. **EXISTS** — проверка существования строк. **Оконные функции** — вычисления над набором строк с сохранением числа строк результата. **CTE (WITH)** — общие табличные выражения для декомпозиции сложных запросов.

Сравнение способов фильтрации.

Способ	Идея	Когда использовать
`IN (SELECT ...)`	Сравнение со списком	Небольшие списки
`EXISTS`	Проверка существования	Большие наборы
`JOIN`	Соединение	Нужны данные обеих таблиц
`LEFT JOIN + IS NULL`	Антисоединение	Поиск «не связанных» записей

GROUP BY vs оконные функции.

Характеристика	GROUP BY	Оконная функция
Число строк	Уменьшается	Сохраняется
Синтаксис	`GROUP BY col`	`OVER (PARTITION BY col)`
Функции	`SUM, AVG, COUNT`	`ROW_NUMBER, RANK, DENSE_RANK, NTILE`
Фильтрация	`HAVING`	Подзапрос / CTE

Основные оконные функции.

Функция	Назначение
`ROW_NUMBER()`	Уникальный номер строки
`RANK()`	Ранг с пропусками при равенстве
`DENSE_RANK()`	Ранг без пропусков
`NTILE(n)`	Разбиение на n групп
`SUM() OVER()`	Сумма / нарастающий итог
`LAG() / LEAD()`	Предыдущая / следующая строка

2.2 Дано (стартовый набор данных)

Дано (стартовый набор данных).

Учебная БД магазина: клиенты, сотрудники, службы доставки, категории, товары, заказы, позиции заказов.

```
CREATE TABLE customers (
    custid      INTEGER PRIMARY KEY,
    companyname VARCHAR(60),
    contactname VARCHAR(60),
    city        VARCHAR(30),
    country     VARCHAR(30)
);

CREATE TABLE employees (
    empid     INTEGER PRIMARY KEY,
    firstname VARCHAR(30),
    lastname  VARCHAR(30),
    city      VARCHAR(30),
    country   VARCHAR(30),
    title     VARCHAR(50)
);

CREATE TABLE shippers (
    shipperid   INTEGER PRIMARY KEY,
    companyname VARCHAR(60)
);

CREATE TABLE categories (
    categoryid   INTEGER PRIMARY KEY,
    categoryname VARCHAR(50)
);

CREATE TABLE products (
    productid    INTEGER PRIMARY KEY,
    productname  VARCHAR(60),
    categoryid   INTEGER REFERENCES categories(categoryid),
    unitprice    NUMERIC(10,2),
    discontinued INTEGER
);

CREATE TABLE orders (
    orderid      INTEGER PRIMARY KEY,
    custid       INTEGER REFERENCES customers(custid),
    empid        INTEGER REFERENCES employees(empid),
    orderdate    DATE,
    requireddate DATE,
    shippeddate  DATE,
    shipperid    INTEGER REFERENCES shippers(shipperid),
    freight      NUMERIC(10,2)
);

CREATE TABLE orderdetails (
    orderid   INTEGER REFERENCES orders(orderid),
    productid INTEGER REFERENCES products(productid),
    unitprice NUMERIC(10,2),
    qty       INTEGER,
    discount  NUMERIC(3,2),
    PRIMARY KEY (orderid, productid)
```

```
);
```

```
INSERT INTO customers VALUES
(1,'Customer A','Ivanov, Petr','Moscow','Russia'),
(2,'Customer B','Petrova, Anna','SPb','Russia'),
(3,'Customer C','Sidorov, Oleg','Berlin','Germany'),
(4,'Customer D','Smirnova, Olga','Madrid','Spain'),
(5,'Customer E','Kuznetsov, Sergey','Paris','France'),
(6,'Customer F','Morozov, Dmitry','Moscow','Russia');

INSERT INTO employees VALUES
(1,'Ivan','Ivanov','Moscow','Russia','Sales Manager'),
(2,'Anna','Petrova','SPb','Russia','Sales Representative'),
(3,'Oleg','Sidorov','Berlin','Germany','Sales Manager'),
(4,'Olga','Smirnova','Madrid','Spain','Sales Representative');

INSERT INTO shippers VALUES
(1,'Speedy Express'),
(2,'United Package'),
(3,'Federal Shipping');

INSERT INTO categories VALUES
(1,'Beverages'),
(2,'Condiments'),
(3,'Confections'),
(8,'Seafood');

INSERT INTO products VALUES
(1,'Chai',1,18.00,0),
(2,'Chang',1,19.00,0),
(3,'Aniseed Syrup',2,10.00,0),
(4,'Chef Anton',2,22.00,1),
(5,'Ikura',8,31.00,0),
(6,'Sir Rodney',3,45.00,0),
(7,'Carnarvon',3,12.00,0);

INSERT INTO orders VALUES
(101,1,1,'2007-01-15','2007-02-15','2007-01-18',1,50.00),
(102,1,1,'2007-02-20','2007-03-20','2007-02-25',1,40.00),
(103,2,2,'2007-05-10','2007-06-10','2007-05-15',2,30.00),
(104,3,3,'2007-07-01','2007-08-01','2007-07-12',2,80.00),
(105,4,4,'2008-03-10','2008-04-10','2008-03-20',3,60.00),
(106,5,1,'2008-05-01','2008-06-01','2008-05-20',1,70.00),
(107,1,2,'2008-05-15','2008-06-15','2008-06-01',3,55.00),
(108,3,3,'2008-09-01','2008-10-01',NULL,2,45.00);

INSERT INTO orderdetails VALUES
(101,1,18.00,10,0.00),(101,3,10.00,5,0.00),
(102,2,19.00,8,0.05),
(103,5,31.00,4,0.00),
(104,6,45.00,2,0.10),(104,7,12.00,10,0.00),
(105,1,18.00,15,0.00),(105,2,19.00,10,0.10),
(106,6,45.00,5,0.15),(106,5,31.00,3,0.00),
```

```
(107,3,10.00,20,0.05),(107,7,12.00,15,0.00),
(108,1,18.00,25,0.00);
```

Сводка по таблицам:

Таблица	Строк	Ключевые поля
customers	6	custid (PK)
employees	4	empid (PK)
shippers	3	shipperid (PK)
categories	4	categoryid (PK)
products	7	productid (PK), categoryid (FK)
orders	8	orderid (PK), custid, empid, shipperid (FK)
orderdetails	13	(orderid, productid) PK

Иллюстрация: клиенты и их заказы.

custid	companyname	Число заказов	Сумма по заказам
1	Customer A	3	80.00
2	Customer B	1	30.00
3	Customer C	2	125.00
4	Customer D	1	60.00
5	Customer E	1	70.00
6	Customer F	0	NULL

2.3 Задачи

1. **4.1.** Клиенты и число их заказов: (а) коррелированный подзапрос; (б) `LEFT JOIN + GROUP BY`.
2. **4.2.** Клиенты без заказов: (а) `NOT EXISTS`; (б) `LEFT JOIN ...IS NULL`.
3. **4.3.** Товары дороже средней цены в своей категории (подзапрос).
4. **4.4.** Топ-3 товара по выручке внутри категории через `DENSE_RANK()`. Выручка: `unitprice*qty*`
5. **4.5.** Для каждого заказа 2008 г.: `orderid`, сумма заказа, % от общей выручки 2008 (`SUM() OVER ()`).
6. **4.6.** CTE: клиент(ы) с максимальной суммарной выручкой.
7. **4.7.** Нарастающий итог продаж по месяцам 2007 г.

Ожидаемый результат:

Столбцы имеют псевдонимы; результаты отсортированы. Сумма выручки по товарам, категориям и клиентам совпадает.

3 Семинар 5. Транзакции, индексирование, статистика, план выполнения

3.1 Теоретическая справка

Теория.

Транзакция — набор операций, выполняемый как единое целое. Свойства **ACID**: Atomicity, Consistency, Isolation, Durability.

Уровни изоляции.

Уровень	Dirty Read	Non-repeatable	Phantom
`READ UNCOMMITTED`	возможно	возможно	возможно
`READ COMMITTED`	нет	возможно	возможно
`REPEATABLE READ`	нет	нет	нет*
`SERIALIZABLE`	нет	нет	нет

* В PostgreSQL `REPEATABLE READ` предотвращает и фантомы.

Типы индексов PostgreSQL.

Индекс	Для чего	Когда
B-Tree	`=, <, >, BETWEEN`, сортировка	Универсальный
Hash	Только `=`	Точечные запросы
GIN	Массивы, JSONB	`@@`, `<@`
GiST	Геометрия, диапазоны	Пространственные данные
BRIN	Упорядоченные большие таблицы	Логи, временные ряды

Методы доступа к данным.

Метод	Когда используется
`Seq Scan`	Большая доля выбираемых строк
`Index Scan`	Малая доля выбираемых строк
`Index Only Scan`	Все столбцы есть в индексе
`Bitmap Heap Scan`	Средняя доля; сначала битовая карта

3.2 Дано (стартовый набор данных)

Дано (стартовый набор данных).

Таблицы `jobs` и `employees` для отработки транзакций, индексов и статистики. Тестовая таблица `OrderDetails_test` создаётся студентом по заданию 5.3.

```sql
CREATE TABLE jobs (
    job_id     VARCHAR(10) PRIMARY KEY,
    job_title  VARCHAR(30) NOT NULL,
    min_salary NUMERIC(8,2) DEFAULT 8000,
    max_salary NUMERIC(8,2),
    CHECK (max_salary <= 25000)
);

CREATE TABLE employees (
    employee_id  INTEGER PRIMARY KEY,
    first_name   VARCHAR(20),
    last_name    VARCHAR(25) NOT NULL,
    email        VARCHAR(25) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    hire_date    DATE NOT NULL,
    job_id       VARCHAR(10) REFERENCES jobs(job_id),
    salary       NUMERIC(8,2)
);
```

```sql
INSERT INTO jobs VALUES
('IT_PROG',  'Programmer',   8000, 20000),
('SA_MAN',   'Sales Manager',10000, 25000),
('SA_REP',   'Sales Rep',    8000, 15000),
('ST_CLERK', 'Stock Clerk',  8000, 12000),
('AC_MGR',   'Accountant',   9000, 22000);

INSERT INTO employees VALUES
(1,'Ivan','Ivanov','ivan.ivanov@st.ru','8(111)111-1111','2020-01-15','IT_PROG',
   12000),
(2,'Anna','Petrova','anna.petrova@st.ru','8(222)222-2222','2021-03-10','SA_REP',
   10000),
(3,'Oleg','Sidorov','oleg.sidorov@st.ru','8(333)333-3333','2019-07-01','SA_MAN',
   20000),
(4,'Olga','Smirnova','olga.smirnova@st.ru','8(444)444-4444','2022-09-20','IT_PROG',
   11000),
(5,'Sergey','Kuznetsov','sergey.k@st.ru','8(555)555-5555','2018-11-05','ST_CLERK',
   9000),
(6,'Dmitry','Morozov','dmitry.m@st.ru','8(666)666-6666','2023-02-01','SA_REP', 9500);
```

Таблица `jobs`**:**

job_id (PK)	job_title	min_salary	max_salary
IT_PROG	Programmer	8000	20000
SA_MAN	Sales Manager	10000	25000
SA_REP	Sales Rep	8000	15000
ST_CLERK	Stock Clerk	8000	12000
AC_MGR	Accountant	9000	22000

Таблица `employees`**:**

employee_id	first_name	last_name	job_id	salary
1	Ivan	Ivanov	IT_PROG	12000
2	Anna	Petrova	SA_REP	10000
3	Oleg	Sidorov	SA_MAN	20000
4	Olga	Smirnova	IT_PROG	11000
5	Sergey	Kuznetsov	ST_CLERK	9000
6	Dmitry	Morozov	SA_REP	9500

3.3 Задачи

1. **5.1.** Явная транзакция: должность в `jobs` + сотрудник в `employees`, `COMMIT`; затем повторить с ошибкой и `ROLLBACK`.

2. **5.2.** Продемонстрировать `READ COMMITTED` в двух сессиях:

   - Сессия 1: `BEGIN`; обновить email сотрудника 1; не закрывать транзакцию.
   - Сессия 2: `BEGIN`; `SELECT` к `employees` (изменения не видны); попробовать обновить email того же сотрудника; зафиксировать.
   - Сессия 1: `COMMIT`. Объяснить результат.

3. **5.3.** Создать `OrderDetails_test`:

```
CREATE TABLE OrderDetails_test AS
SELECT round(random()*100) as order_id,
       'Product '|| round(random()*100)::char(3) as product_name,
       random()*100::money as price,
       order_date
FROM generate_series('2022-01-01','2023-12-31', INTERVAL '1 day') as order_date,
     generate_series(1, 5) as id;
```

4. **5.4.** Заказы за 22.01.2022. План через `EXPLAIN ANALYZE`. Какой метод доступа?

5. **5.5.** B-Tree индекс на `order_date`. Повторить запрос. Какой метод доступа?

6. **5.6.** Hash-индекс на `order_date`. Повторить. Какой индекс использовался?

7. **5.7.** Кардинальность для `OrderDetails_2` (1 день) и `OrderDetails_3` (1 час); `default_statistic` = 10; `ANALYZE`; оценка для Product 60.

Ожидаемый результат:

Планы до/после индексов, объяснение метода доступа и кардинальности.

4 Семинар 6. Проектирование БД, нормализация, DDL, ограничения, функции и триггеры

4.1 Теоретическая справка

Теория.

Нормализация: 1НФ — атомарность; 2НФ — полная зависимость от ключа; 3НФ — нет транзитивных зависимостей; НФБК — каждый детерминант — ключ.

Сводка нормальных форм.

НФ	Требование	Что устраняет
1НФ	Атомарные значения	Повторяющиеся группы
2НФ	1НФ + полная зависимость от ключа	Частичные зависимости
3НФ	2НФ + нет транзитивных зависимостей	Зависимости неключевых атрибутов
НФБК	3НФ + каждый детерминант — ключ	Аномалии, не устраняемые 3НФ

Ограничения целостности.

Ограничение	Назначение	Пример
`PRIMARY KEY`	Уникальная идентификация	`id INT PRIMARY KEY`
`FOREIGN KEY`	Ссылочная целостность	`REFERENCES jobs`
`UNIQUE`	Уникальность	`email VARCHAR UNIQUE`
`NOT NULL`	Обязательность	`name VARCHAR NOT NULL`
`CHECK`	Доменное ограничение	`CHECK (age>=18)`
`DEFAULT`	Значение по умолчанию	`status DEFAULT 'new'`

Опции FK.

Опция	Поведение
`NO ACTION`	Запрет (по умолчанию)
`RESTRICT`	Запрет (проверка сразу)
`CASCADE`	Каскадное удаление / обновление
`SET NULL`	Установка NULL в FK
`SET DEFAULT`	Установка DEFAULT в FK

Иллюстрация: приведение к 1НФ. Исходная таблица (нарушение 1НФ — множественные телефоны в одной ячейке):

Сотрудник	Отдел	Телефоны
Иванов И.И.	IT	286-54-94, 765-68-43
Петров С.Ю.	HR	286-55-97

После приведения к 1НФ:

Сотрудник	Отдел	Телефон
Иванов И.И.	IT	286-54-94
Иванов И.И.	IT	765-68-43
Петров С.Ю.	HR	286-55-97

4.2 Дано (стартовый набор данных)

Дано (стартовый набор данных).

Часть А. Ненормализованная таблица `orders_raw` — для задачи 6.1.
Часть Б. Схема `schema1` — для задач 6.3–6.6.

Часть А. Ненормализованная таблица (нарушает 1НФ):

№ заказа	Клиент	Адрес	Товар	Кол-во	Цена	Дата
12345	Иванов И.И.	ул. Ленина, 15	Смартфон	1	25000	10.11.2023
12345	Иванов И.И.	ул. Ленина, 15	Стекло	1	1000	10.11.2023
12346	Петрова М.П.	ул. Мира, 20	Наушники	2	2500	11.11.2023
12347	Сидоров А.Ю.	ул. Ленина, 15	Часы	1	10000	12.11.2023

```sql
-- Raw "flat" table (violates 1NF)
CREATE TABLE orders_raw (
    order_num   INTEGER,
    customer    VARCHAR(100),
    address     VARCHAR(150),
    city        VARCHAR(50),
    product     VARCHAR(50),
    qty         INTEGER,
    price       NUMERIC(10,2),
    order_date  DATE
);

INSERT INTO orders_raw VALUES
(12345,'Ivanov Ivan','ul. Lenina, 15','Moscow','Smartphone',  1,25000,'2023-11-10'),
(12345,'Ivanov Ivan','ul. Lenina, 15','Moscow','Screen glass',1, 1000,'2023-11-10'),
(12346,'Petrova Maria','ul. Mira, 20','Smolensk','Headphones',2, 2500,'2023-11-11'),
(12347,'Sidorov Alexey','ul. Lenina, 15','Moscow','Watch',    1,10000,'2023-11-12');
```

Часть Б. Схема `schema1`:

```sql
CREATE SCHEMA schema1;

CREATE TABLE schema1.jobs (
    job_id     VARCHAR(10) PRIMARY KEY,
    job_title  VARCHAR(30) NOT NULL,
    min_salary NUMERIC(8,2) DEFAULT 8000,
    max_salary NUMERIC(8,2)
);

CREATE TABLE schema1.countries (
    country_id   INTEGER PRIMARY KEY,
    country_name VARCHAR(30),
    region_id    INTEGER
```

```
);

CREATE TABLE schema1.addresses (
    address_id  NUMERIC(4,0),
    street      VARCHAR(40),
    postal_code VARCHAR(6),
    city        VARCHAR(30),
    country_id  INTEGER
);

CREATE TABLE schema1.departments (
    department_id   NUMERIC(4,0) NOT NULL,
    department_name VARCHAR(30) NOT NULL,
    manager_id      NUMERIC(6,0) NOT NULL,
    address_id      NUMERIC(4,0)
);

CREATE TABLE schema1.employees (
    employee_id   NUMERIC(6,0) PRIMARY KEY,
    first_name    VARCHAR(20),
    last_name     VARCHAR(25) NOT NULL,
    email         VARCHAR(25) NOT NULL UNIQUE,
    phone_number  VARCHAR(20),
    hire_date     DATE NOT NULL,
    job_id        VARCHAR(10),
    salary        NUMERIC(8,2),
    manager_id    NUMERIC(6,0),
    department_id NUMERIC(4,0)
);
```

```
INSERT INTO schema1.jobs VALUES
('IT_PROG', 'Programmer',   8000, 20000),
('SA_MAN',  'Sales Manager',10000,25000),
('SA_REP',  'Sales Rep',    8000, 15000);

INSERT INTO schema1.countries VALUES
(1,'Russia',1),(3,'India',2),(5,'China',3);

INSERT INTO schema1.addresses VALUES
(1,'Lenina st, 15','101000','Moscow',  1),
(2,'Mira st, 20',  '214000','Smolensk',1),
(3,'Delhi rd, 5',  '110001','Delhi',   3);

INSERT INTO schema1.departments VALUES
(10,'IT',       1001,1),
(20,'Sales',    1002,1),
(30,'Logistics',1003,2);

INSERT INTO schema1.employees VALUES
(1001,'Ivan','Ivanov','ivan@st.ru','8(111)111-1111','2020-01-15','IT_PROG',12000,NULL
    ,10),
(1002,'Anna','Petrova','anna@st.ru','8(222)222-2222','2021-03-10','SA_REP',
    10000,1001,20),
```

```
(1003,'Oleg','Sidorov','oleg@st.ru','8(333)333-3333','2019-07-01','SA_MAN', 20000,
   NULL,20),
(1004,'Olga','Smirnova','olga@st.ru','8(444)444-4444','2022-09-20','IT_PROG'
   ,11000,1001,10);
```

4.3 Задачи

1. **6.1.** Привести `orders_raw` к 1НФ, 2НФ, 3НФ (DDL-скрипты).
2. **6.2.** Построить ER-модель для «Аренды жилья» (5+ таблиц, 2НФ+, массив, ограничения PK/FK/CHECK/UNIQUE/NOT NULL), преобразовать в реляционную модель.
3. **6.3.** Донастроить `schema1`:
   - `countries.country_id` — PK, автогенерация с шагом 2;
   - `addresses.postal_code` — только цифры (`CHECK`);
   - `employees.email` — UNIQUE, NOT NULL;
   - `employees.phone_number` — шаблон `8(XXX)XXX-XXXX`;
   - `jobs.max_salary` ≤ 25000;
   - `jobs.min_salary` DEFAULT 8000;
   - `employees.job_id` — FK на `jobs` с `ON DELETE CASCADE`;
   - составной FK (`department_id`, `manager_id`) на `departments`.
4. **6.4.** Функции/процедуры:
   (a) скалярная функция «дата → число дней до сегодня»;
   (b) функция «город → список подразделений и число сотрудников»;
   (c) процедура обновления телефона по ID с проверками.
5. **6.5.** Триггеры: `check_salary` (рост $> 100\%$ — исключение), триггер `BEFORE UPDATE` на `employees`, проверка срабатывания, отключение, переименование.
6. **6.6.** DML: добавление записей, нарушение ограничений, работа с массивами телефонов.

Ожидаемый результат:

В отчёте привести SQL-скрипты, скриншоты, объяснение поведения ограничений и триггеров.